MODUEL -- 1

DSA MATHEMATICAL BASICS

Mathematical problems are some of the most common building blocks in DSA. Before moving to more advanced algorithms, you should be comfortable with:

- Divisors
- Counting divisors
- Optimizing divisor problems
- GCD / HCF
- Euclidean Algorithm
- Armstrong numbers
- Extracting digits
- Reversing numbers
- Palindrome numbers

The goal is not just to memorize code. You should understand **why the code works**, what each operator does, and how to identify the correct approach during an interview or coding problem.

---

1. DIVISORS

1.1 Definition

A **divisor** of a number is a number that divides it exactly, leaving a remainder of 0.

In other words:

If n % i == 0, then i is a divisor of n.

Example

Consider:

36 ÷ 4 = 9

There is no remainder.

Therefore:

36 % 4 = 0

So, **4 is a divisor of 36**.

Now consider:

36 ÷ 5 = 7 remainder 1

Therefore:

36 % 5 = 1

So, **5 is not a divisor of 36**.

1.2 Divisors of 36

Let's find all numbers that divide 36 exactly:

1, 2, 3, 4, 6, 9, 12, 18, 36

Each of these numbers satisfies:

36 % divisor == 0

For example:

36 % 1 = 0

36 % 2 = 0

36 % 3 = 0

36 % 4 = 0

36 % 6 = 0

...

36 % 36 = 0

1.3 Basic Method to Find Divisors

The simplest approach is to check every number from 1 to n.

Code

```
n = 36


for i in range(1, n + 1):
    if n % i == 0:
        print(i)
```

How does it work?

Suppose:

n = 36

The loop checks:

i = 1

i = 2

i = 3

i = 4

...

i = 36

For each i, we ask:

n % i == 0

If the answer is True, i is a divisor.

1.4 Important Operators

% — Modulo Operator

The % operator gives the **remainder**.

Example:

36 % 5 = 1

Because:

36 = 5 × 7 + 1

Therefore, remainder = 1.

// — Integer Division

The // operator gives the **quotient without the decimal part**.

Example:

36 // 5 = 7

Because:

36 ÷ 5 = 7.2

The integer quotient is 7.

1.5 Key Point to Remember

The most important condition for checking whether i is a divisor of n is:

n % i == 0

Think:

Divides exactly → remainder must be 0.

1.6 Time Complexity

The loop checks every number from 1 to n.

Therefore:

Time Complexity = O(n)

Space Complexity = O(1)

---

2. COUNTING DIVISORS

Finding divisors and counting divisors are almost the same problem.

Instead of printing every divisor, we maintain a counter.

2.1 Idea

Start with:

count = 0

Whenever a divisor is found:

count = count + 1

At the end, count contains the total number of divisors.

2.2 Example

Find the number of divisors of 36.

The divisors are:

1, 2, 3, 4, 6, 9, 12, 18, 36

There are:

9 divisors

Therefore:

Answer = 9

2.3 Code

```
n = 36
count = 0

for i in range(1, n + 1):
    if n % i == 0:
        count = count + 1

print(count)
```

Output:

9

2.4 Understanding the Important Lines

count = 0

This initializes the counter.

Initially, we have not found any divisor.

count = 0

---

if n % i == 0

This checks whether i divides n.

If true, we found a divisor.

---

count = count + 1

Increase the counter by one.

For example:

First divisor found → count = 1

Second divisor found → count = 2

Third divisor found → count = 3

...

2.5 Key Point to Remember

Whenever a problem says:

"Count how many numbers satisfy a condition"

A common pattern is:

count = 0

```
for ...:
    if condition:
        count += 1
```

For counting divisors, the condition is:

n % i == 0

2.6 Time Complexity

Time Complexity = O(n)

Space Complexity = O(1)

---

3. OPTIMIZED METHOD TO FIND DIVISORS

The previous method checks every number from:

1 → n

But this is unnecessary.

We can solve the problem in:

O(√n)

by understanding an important property of divisors.

3.1 Divisors Occur in Pairs

Consider:

36

Its factor pairs are:

1 × 36 = 36

2 × 18 = 36

3 × 12 = 36

4 × 9 = 36

6 × 6 = 36

Notice something important.

Whenever we find one divisor, we automatically know its corresponding pair.

For example:

4 × 9 = 36

If we discover:

4

we immediately know:

9

because:

36 // 4 = 9

3.2 Why Only Check Until √n?

This is the important mathematical idea.

Suppose:

a × b = n

If both a and b were greater than √n, then:

a × b > √n × √n

Therefore:

a × b > n

But we know:

a × b = n

This is impossible.

Therefore, in every divisor pair, **at least one divisor must be less than or equal to √n**.

So we only need to check:

1 → √n

and find the paired divisor using:

n // i

3.3 Example with 36

√36 = 6

So we only check:

1, 2, 3, 4, 5, 6

We don't need to check:

7, 8, 9, ..., 36

because their corresponding smaller divisors have already been checked.

3.4 Optimized Code

```
n = 36


for i in range(1, int(n ** 0.5) + 1):


    if n % i == 0:


        print(i)


        if i != n // i:
            print(n // i)
```

3.5 Understanding the Code

n ** 0.5

This calculates the square root of n.

For example:

36 ** 0.5 = 6

---

int(n ** 0.5)

Converts the result into an integer.

For example:

int(6.0) = 6

---

range(1, int(n ** 0.5) + 1)

We add +1 because Python's range() excludes the ending value.

For example:

range(1, 7)

produces:

1, 2, 3, 4, 5, 6

Therefore:

range(1, int(n ** 0.5) + 1)

allows us to include √n.

---

if n % i == 0

Check whether i is a divisor.

---

n // i

Find the paired divisor.

Example:

36 // 4 = 9

So when we find 4, we also find 9.

3.6 Why Do We Need i != n // i?

Consider:

36

One divisor pair is:

6 × 6 = 36

When:

i = 6

we get:

n // i = 36 // 6 = 6

So if we simply print both:

```
print(i)
print(n // i)
```

we would get:

6

6

The same divisor would be printed twice.

Therefore we use:

```
if i != n // i:
    print(n // i)
```

This prevents duplicate printing for perfect squares.

3.7 Important Special Case: Perfect Squares

A **perfect square** is a number whose square root is an integer.

Examples:

1 = 1 × 1

4 = 2 × 2

9 = 3 × 3

16 = 4 × 4

25 = 5 × 5

36 = 6 × 6

For perfect squares, the middle divisor pairs with itself.

Example:

36

Pair:

6 × 6

Therefore, it should only be counted once.

3.8 Time Complexity

Basic method:

O(n)

Optimized method:

O(√n)

Space:

O(1)

Key Point to Remember

Divisors occur in pairs, so check only up to √n and obtain the other divisor using n // i.

The pattern to remember is:

```
if n % i == 0:
    divisor1 = i
    divisor2 = n // i
```

And for perfect squares:

```
if i != n // i:
```

---

4. GCD / HCF

4.1 Full Forms

GCD = Greatest Common Divisor

HCF = Highest Common Factor

GCD and HCF mean the same thing.

4.2 Definition

The **GCD of two numbers is the largest positive integer that divides both numbers exactly.**

4.3 Example: GCD of 12 and 18

Divisors of 12:

1, 2, 3, 4, 6, 12

Divisors of 18:

1, 2, 3, 6, 9, 18

Common divisors:

1, 2, 3, 6

The greatest one is:

6

Therefore:

GCD(12, 18) = 6

---

5. BASIC METHOD TO FIND GCD

5.1 Idea

We can check every number from:

1 → smaller number

For each number, check whether it divides both numbers.

If it does, update the GCD.

5.2 Code

```
a = 12
b = 18

gcd = 1

for i in range(1, min(a, b) + 1):

    if a % i == 0 and b % i == 0:
```

```
        gcd = i

print("GCD/HCF =", gcd)
```

Output:

GCD/HCF = 6

5.3 Understanding the Condition

a % i == 0 and b % i == 0

There are two conditions:

a % i == 0

means i divides a.

And:

b % i == 0

means i divides b.

The and means **both conditions must be true**.

Therefore, i must be a common divisor.

5.4 Why min(a, b)?

Suppose:

a = 12

b = 18

The GCD cannot be greater than 12, because 12 is the smaller number.

Therefore:

range(1, min(a, b) + 1)

is sufficient.

5.5 Why Does gcd = i Work?

We check numbers from smaller to larger:

1, 2, 3, 4, 5, ..., 12

Whenever we find a common divisor, we replace gcd.

For 12 and 18:

i = 1 → common → gcd = 1

i = 2 → common → gcd = 2

i = 3 → common → gcd = 3

i = 6 → common → gcd = 6

No larger common divisor exists.

Therefore:

gcd = 6

5.6 Time Complexity

O(min(a, b))

Space:

O(1)

Key Point to Remember

For the basic GCD method, check every number up to the smaller number and keep updating the largest common divisor found.

---

6. EUCLIDEAN ALGORITHM

The basic GCD method works, but it can be inefficient for large numbers.

The **Euclidean Algorithm** provides a much faster method.

6.1 Main Idea

The fundamental rule is:

GCD(a, b) = GCD(b, a % b)

We repeatedly calculate the remainder until the remainder becomes 0.

When that happens, the other number is the GCD.

6.2 Example: GCD(18, 12)

Start with:

a = 18

b = 12

Step 1

Calculate:

18 % 12 = 6

Therefore:

GCD(18, 12) = GCD(12, 6)

Step 2

Calculate:

12 % 6 = 0

Therefore:

GCD(12, 6) = 6

Since the remainder is now 0, we stop.

Answer:

GCD = 6

6.3 Code

```
a = 18
b = 12

while b != 0:
    a, b = b, a % b

print("GCD/HCF =", a)
```

Output:

GCD/HCF = 6

6.4 Understanding a, b = b, a % b

This is one of the most important lines.

a, b = b, a % b

It means:

new a = old b

new b = old a % old b

Python evaluates the right-hand side first.

For example:

a = 18

b = 12

Calculate:

b = 12

a % b = 18 % 12 = 6

Then:

a = 12

b = 6

6.5 Complete Dry Run

Initial:

a = 18

b = 12

Iteration 1

a % b

18 % 12 = 6

Update:

a = 12

b = 6

Iteration 2

12 % 6 = 0

Update:

a = 6

b = 0

Loop condition

while b != 0:

But:

b = 0

Therefore, the loop stops.

Final:

a = 6

b = 0

So:

GCD = 6

6.6 Why Does the Algorithm Work?

The important mathematical property is:

GCD(a, b) = GCD(b, a % b)

Suppose:

a = bq + r

where r = a % b.

Any number that divides both a and b must also divide:

a - bq = r

Therefore, the common divisors do not change when we replace:

(a, b)

with:

(b, a % b)

This allows us to reduce the problem repeatedly until the remainder becomes zero.

6.7 Time Complexity

The Euclidean Algorithm runs in:

O(log(min(a, b)))

This is significantly faster than:

O(min(a, b))

for the basic method.

Space:

O(1)

when implemented iteratively.

Key Point to Remember

Memorize the process, not just the code:

Divide → take remainder → replace → repeat until remainder becomes 0.

The formula is:

GCD(a, b) = GCD(b, a % b)

And the standard Python implementation is:

```
while b != 0:
    a, b = b, a % b
```

When b == 0:

a = GCD

---

7. ARMSTRONG NUMBER

7.1 Definition

A number is called an **Armstrong number** if the sum of each digit raised to the power of the total number of digits is equal to the original number.

For a number containing k digits:

Armstrong condition:

$$digit_1^k + digit_2^k + ... + digit_k^k = original\ number$$

7.2 Example: 153

The number:

153

contains:

3 digits

Therefore, each digit must be raised to the power 3.

$1^3 + 5^3 + 3^3$

Calculate:

1 + 125 + 27

Therefore:

153

The result is equal to the original number.

Hence:

153 is an Armstrong number.

7.3 General Formula

For a number with k digits:

number = $d_1d_2d_3...d_k$

The Armstrong condition is:

$d_1^k + d_2^k + d_3^k + ... + d_k^k$ = number

7.4 Important Note About 3-Digit Problems

Many beginner exercises specifically ask:

"Check whether a three-digit number is an Armstrong number."

For three-digit numbers, the power is always:

3

So we use:

digit ** 3

However, the **general Armstrong definition** uses:

number of digits

as the power.

7.5 Extracting Digits

To solve Armstrong problems using arithmetic, we need two important operations.

Extract the last digit

n % 10

Example:

153 % 10 = 3

Therefore, the last digit is:

3

Remove the last digit

n // 10

Example:

153 // 10 = 15

So:

153 → 15 → 1 → 0

This allows us to process every digit one by one.

7.6 Code for a Three-Digit Armstrong Number

```
n = 153

original = n
digit_sum = 0

while n > 0:

    lastdigit = n % 10

    digit_sum = digit_sum + lastdigit ** 3

    n = n // 10

if digit_sum == original:
    print("It is an Armstrong number")
else:
    print("Not Armstrong")
```

Output:

It is an Armstrong number

7.7 Understanding the Code

original = n

This is extremely important.

Initially:

n = 153

During the loop, we repeatedly remove the last digit.

Eventually:

n = 0

Therefore, we cannot compare the final n with the Armstrong sum.

We save the original value:

original = n

Then compare:

digit_sum == original

---

digit_sum = 0

This stores the sum of the powered digits.

Initially:

digit_sum = 0

---

lastdigit = n % 10

Extracts the last digit.

For:

n = 153

we get:

153 % 10 = 3

---

digit_sum = digit_sum + lastdigit ** 3

Add the cube of the digit.

For the first digit:

$3^3 = 27$

So:

digit_sum = 27

---

n = n // 10

Remove the last digit.

153 // 10 = 15

Then:

15 // 10 = 1

Then:

1 // 10 = 0

7.8 Complete Dry Run of 153

Initial:

n = 153

original = 153

digit_sum = 0

Iteration 1

Last digit:

153 % 10 = 3

Add cube:

$3^3 = 27$

So:

digit_sum = 27

Remove digit:

153 // 10 = 15

Iteration 2

Last digit:

15 % 10 = 5

Add cube:

$5^3 = 125$

So:

digit_sum = 27 + 125

= 152

Remove digit:

15 // 10 = 1

Iteration 3

Last digit:

1 % 10 = 1

Add cube:

$1^3 = 1$

So:

digit_sum = 153

Remove digit:

1 // 10 = 0

Loop stops.

Now:

digit_sum = 153

original = 153

Therefore:

digit_sum == original

So 153 is an Armstrong number.

7.9 Common Armstrong Mistake

Incorrect:

if digit_sum == n:

Why is this wrong?

Because after the loop:

n = 0

Correct:

original = n

and later:

if digit_sum == original:

Key Points to Remember

For digit-based problems, remember:

n % 10 → extract last digit

n // 10 → remove last digit

For Armstrong:

1. Save original number.
2. Extract each digit.
3. Raise digit to required power.
4. Add it to the sum.
5. Remove the digit.

6. Compare the sum with original.

The most important memory pattern is:

original = n

```
while n > 0:
    digit = n % 10
    # process digit
    n = n // 10

# compare result with original
```

---

8. EXTRACTING EVERY DIGIT OF A NUMBER

Digit extraction is not a separate trick only for Armstrong numbers.

It is a fundamental pattern used in many DSA problems.

8.1 Problem

Given:

12345

Print every digit.

Expected output:

5

4

3

2

1

Notice that arithmetic extraction processes digits from **right to left**.

8.2 Code

```
n = 12345

while n > 0:

    digit = n % 10
    print(digit)
```

```
    n = n // 10
```

Output:

5

4

3

2

1

8.3 Why Does It Work?

Start:

n = 12345

Extract:

12345 % 10 = 5

Remove:

12345 // 10 = 1234

Then:

1234 % 10 = 4

1234 // 10 = 123

Continue until:

n = 0

Key Point to Remember

Memorize:

n % 10 → last digit

n // 10 → remove last digit

This pattern appears repeatedly in number-based DSA problems.

---

9. REVERSE A NUMBER

9.1 Problem

Given:

12345

Reverse it:

54321

9.2 Main Idea

We repeatedly:

1. Extract the last digit.
2. Add it to the reversed number.
3. Remove the last digit.

The important formula is:

reverse = reverse × 10 + digit

9.3 Code

```
n = 12345

reverse = 0

while n > 0:
    digit = n % 10
    reverse = reverse * 10 + digit
    n = n // 10

print(reverse)
```

Output:

54321

9.4 Dry Run

Initial:

n = 12345

reverse = 0

Step 1

Extract:

digit = 5

Update:

reverse = 0 × 10 + 5

= 5

Remove:

n = 1234

Step 2

digit = 4

Update:

reverse = 5 × 10 + 4

= 54

Step 3

digit = 3

reverse = 54 × 10 + 3

= 543

Step 4

digit = 2

reverse = 543 × 10 + 2

= 5432

Step 5

digit = 1

reverse = 5432 × 10 + 1

= 54321

Final:

reverse = 54321

9.5 Why Multiply by 10?

Suppose:

reverse = 54

and the next digit is:

3

We want:

543

Multiplying by 10 shifts the existing digits left:

54 × 10 = 540

Then add 3:

540 + 3 = 543

Therefore:

reverse = reverse × 10 + digit

Key Point to Remember

For reversing a number:

digit = n % 10

reverse = reverse * 10 + digit

n = n // 10

This is one of the most important number-manipulation patterns in DSA.

---

10. PALINDROME NUMBER

10.1 Definition

A number is called a **palindrome** if it remains the same when reversed.

Examples:

121 → 121

1331 → 1331

444 → 444

These are palindromes.

Non-examples:

123 → 321

1234 → 4321

These are not palindromes.

10.2 Main Idea

To check whether a number is a palindrome:

1. Store the original number.
2. Reverse the number.
3. Compare the reversed number with the original.

If:

reverse == original

then it is a palindrome.

10.3 Code

n = 121

```
original = n
reverse = 0

while n > 0:

    digit = n % 10

    reverse = reverse * 10 + digit

    n = n // 10

if reverse == original:
    print("Palindrome")
else:
    print("Not Palindrome")
```

Output:

Palindrome

10.4 Dry Run

Original:

121

Reverse process:

1

12

121

At the end:

original = 121

reverse = 121

Therefore:

reverse == original

So:

121 is a palindrome.

Key Point to Remember

Palindrome problems usually follow this pattern:

Save original

↓

Reverse number

↓

Compare

↓

Same → Palindrome

Different → Not Palindrome

---

11. COMMON MISTAKES

Mistake 1: Using // instead of % for divisibility

Incorrect:

```
if n // i == 0:
```

Correct:

```
if n % i == 0:
```

Remember:

% → remainder

// → quotient

Mistake 2: Forgetting +1 in range()

Incorrect:

```
range(1, int(n ** 0.5))
```

Correct:

```
range(1, int(n ** 0.5) + 1)
```

Python excludes the ending value.

Mistake 3: Printing the square-root divisor twice

For:

36

we have:

6 × 6 = 36

Therefore, use:

```
if i != n // i:
```

Mistake 4: Comparing Armstrong sum with modified n

Incorrect:

if digit_sum == n:

After the loop:

n = 0

Correct:

original = n

and:

if digit_sum == original:

Mistake 5: Forgetting to initialize variables

For example:

digit_sum = 0

reverse = 0

count = 0

These variables need a starting value before the loop uses them.

Mistake 6: Confusing n % 10 and n // 10

Remember:

n % 10

↓

Extract last digit

Example:

12345 % 10 = 5

And:

n // 10

↓

Remove last digit

Example:

12345 // 10 = 1234

---

12. IMPORTANT PATTERNS TO MEMORIZE

Pattern 1: Check Divisibility

if n % i == 0:

Meaning:

i divides n exactly.

Pattern 2: Extract Last Digit

digit = n % 10

Meaning:

Get the last digit.

Pattern 3: Remove Last Digit

n = n // 10

Meaning:

Remove the last digit.

Pattern 4: Process Every Digit

```
while n > 0:

    digit = n % 10

    # process digit

    n = n // 10
```

This pattern is useful for:

- Armstrong number
- Sum of digits
- Count digits
- Reverse number
- Palindrome
- Product of digits
- Digit frequency

Pattern 5: Reverse Number

```
reverse = 0

while n > 0:

    digit = n % 10
```

reverse = reverse * 10 + digit

n = n // 10

Pattern 6: Save Original Number

Whenever the algorithm modifies n but you need the original value later:

original = n

This is especially important for:

- Armstrong
- Palindrome
- Reverse comparison
- Other digit-based problems

---

13. COMPLEXITY SUMMARY

Problem	Time Complexity	Space Complexity
Find divisors — basic	O(n)	O(1)
Count divisors — basic	O(n)	O(1)
Find divisors — optimized	O(√n)	O(1)
GCD — basic	O(min(a, b))	O(1)
GCD — Euclidean Algorithm	O(log(min(a, b)))	O(1)
Armstrong number	O(d)	O(1)
Extract digits	O(d)	O(1)
Reverse number	O(d)	O(1)
Palindrome number	O(d)	O(1)

Here:

d = number of digits

For example:

12345 → d = 5

---

14. QUICK REVISION TABLE

Concept	Core Idea	Pattern to Remember
Divisor	Divides exactly	n % i == 0

Concept	Core Idea	Pattern to Remember
Count Divisors	Count every divisor	count += 1
Divisor Pair	Every divisor has a partner	n // i
Optimized Divisors	Check only up to √n	i <= √n
Perfect Square	Middle divisor pairs with itself	i == n // i
GCD / HCF	Greatest common divisor	Common divisor
Euclidean Algorithm	Repeated remainder	a, b = b, a % b
Last Digit	Extract final digit	n % 10
Remove Digit	Remove final digit	n // 10
Armstrong	Sum powered digits	digit ** k
Reverse	Build number from digits	reverse * 10 + digit
Palindrome	Original equals reverse	original == reverse

---

15. PRACTICE QUESTIONS

Level 1 — Divisors

Question 1

Print all divisors of:

24

Expected output:

1, 2, 3, 4, 6, 8, 12, 24

Question 2

Count the number of divisors of:

48

Question 3

Print all divisors of:

36

using the optimized O(√n) method.

Remember:

n // i

gives the paired divisor.

Level 2 — GCD

Question 4

Find the GCD/HCF of:

12 and 18

using the basic method.

Expected answer:

6

Question 5

Find the GCD/HCF of:

18 and 12

using the Euclidean Algorithm.

Use:

GCD(a, b) = GCD(b, a % b)

Level 3 — Armstrong

Question 6

Check whether:

153

is an Armstrong number.

Expected:

Armstrong

Question 7

Check whether:

370

is an Armstrong number.

Hint:

$3^3 + 7^3 + 0^3$

Level 4 — Digit Manipulation

Question 8

Extract and print every digit of:

12345

Remember:

digit = n % 10

n = n // 10

Question 9

Reverse:

12345

Expected:

54321

Question 10

Check whether a number is a palindrome.

Test your program with:

121

and:

123

Expected:

121 → Palindrome

123 → Not Palindrome

---

16. FINAL MEMORY SHEET

Before moving to more advanced DSA problems, make sure these patterns are automatic in your mind.

Divisibility

n % i == 0

Means:

i divides n exactly.

Last Digit

n % 10

Means:

Get the last digit.

Remove Last Digit

n // 10

Means:

Remove the last digit.

Divisor Pair

n // i

If:

n % i == 0

then:

i

and:

n // i

are a divisor pair.

Square Root Optimization

Instead of:

1 → n

check:

1 → √n

because divisors occur in pairs.

GCD

Basic:

Check common divisors.

Optimized:

GCD(a, b) = GCD(b, a % b)

Code:

```
while b != 0:
    a, b = b, a % b
```

Digit Processing

```
while n > 0:

    digit = n % 10

    # process digit

    n = n // 10
```

Reverse

reverse = reverse * 10 + digit

Armstrong

Sum of each digit raised to the number of digits

= original number

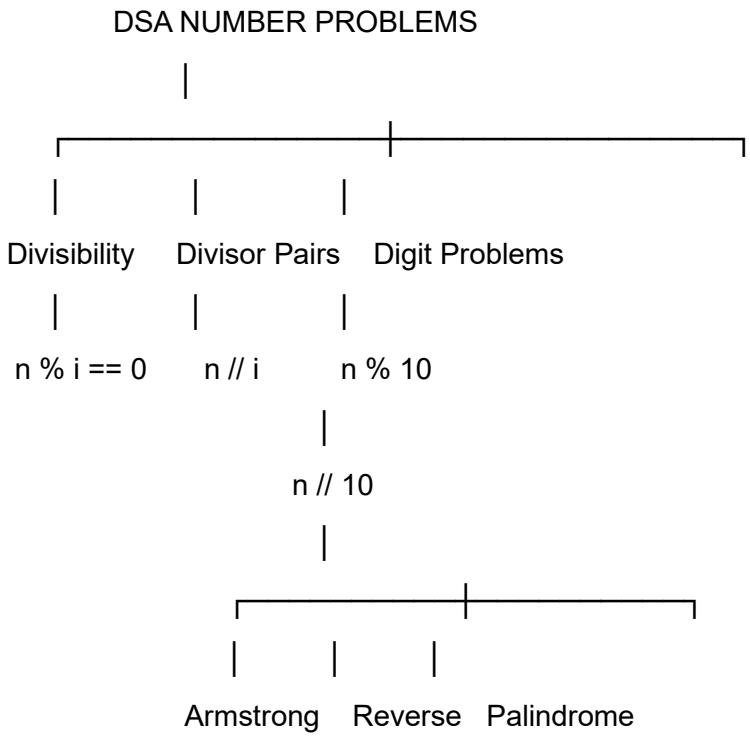
Palindrome

original == reverse

MOST IMPORTANT TAKEAWAY

Do not try to memorize every program separately.

Instead, recognize the **small mathematical patterns** underneath them:



Once these patterns become comfortable, many number-based DSA problems stop looking like completely different questions. They become combinations of the same few ideas.